THE LAWS OF SOFTWARE TEAMS

The 90-90 rule

The last 10% of the code takes the other 90% of the time

swipe →

Ivan Rykov

01 THE HOOK

90% done
Still 90% done

FEATURE · NEW CHECKOUT

WEEK 4

90%

done

no change for two weeks

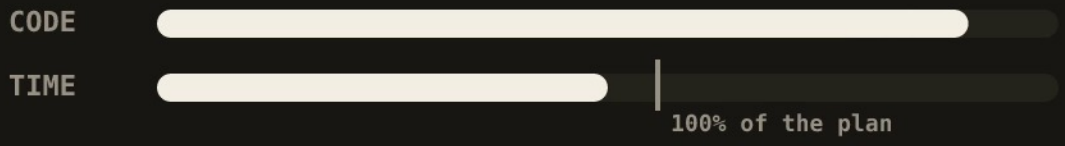
This pattern has a rule

illustration

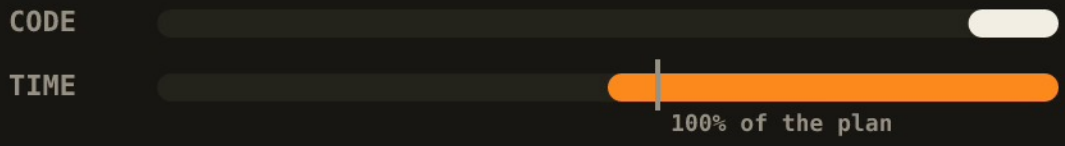
The 90-90 rule

Tom Cargill · Bell Labs

> “The first 90 percent of the code accounts for the first 90 percent of the development time



> The remaining 10 percent of the code accounts for the other 90 percent of the development time”



90% + 90% = 180%



JON BENTLEY · 1985

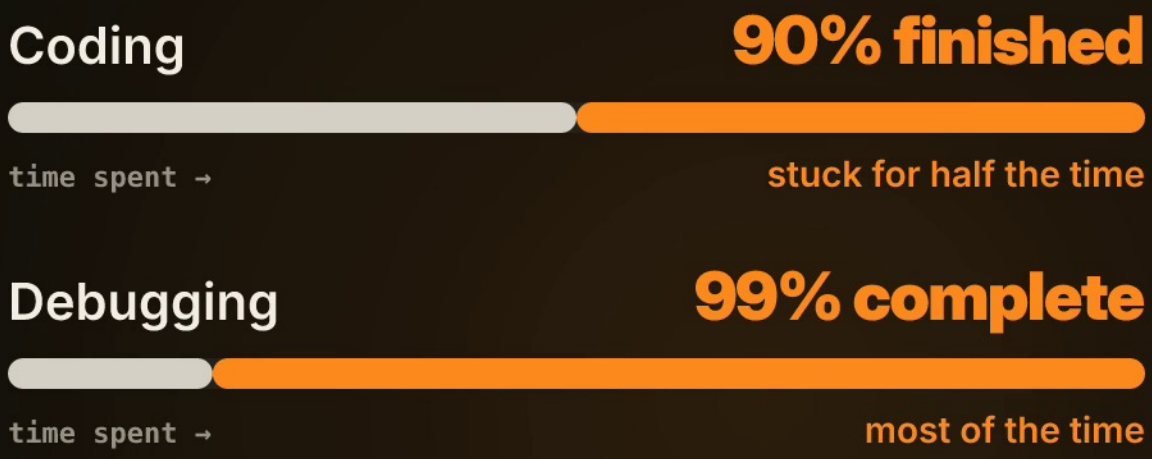
"Programming Pearls", Communications of the ACM: the "Rule of Credibility"

Teams still recognise it

Brooks saw it in 1975

THE MYTHICAL MAN-MONTH · 1975

Fred Brooks · ch. 14, Hatching a Catastrophe



"Coding ... is '90 percent finished' for half of the total coding time. Debugging is '99 percent complete' most of the time"
bars illustrative

Bad news waits until the end

Two studies of large government projects



A too-short estimate does not move
Then it moves all at once

King et al., 1967, as cited by Brooks (ch. 14) · chart illustrative

The last 10%
is not the main code

main code

INSIDE THE LAST 10%

- edge cases
- integration
- data migration
- monitoring
- documentation
- rollout

typical examples

Plan the finish from day one

1. **Plan the last 10% on day one**
   edge cases, migration and rollout go into the plan
2. **Show working slices**
   finished parts, not a percentage
3. **Define done before you start**
   in production and used, not “code complete”

FRED BROOKS · 1975

“Milestones must be concrete, specific, measurable events, defined with knife-edge sharpness”

Done is not 90%

~~90% done~~

Done means:

- it is in production
- people use it

WHAT I TAKE FROM THIS

I plan the finish from the first day:

1. We split big work by feature, not only by stage (code, review, test)
2. Where it makes sense, each part is reviewed and tested on its own: the main flow first, then the main edge cases from the definition of done. Then we test the parts together
3. We try to follow the definition of done and not add new requests. If a real problem comes up during the work, we fix it, and this can change the timeline
4. Integrations with external systems can go in parallel with the core, but it depends. We need to agree on the interface early and be sure that a change to it will not mean redoing the whole integration

WHAT I TAKE FROM THIS

5. **We measure progress by finished parts and their size, not by the number of tasks. The time left is a separate estimate, because people who finish their parts can join the remaining ones**
6. **Big work always has a buffer for the finish, and our timeline includes it. The last stage of the timeline is part of the definition of done: debugging, integration and end-to-end tests, and checks in production and, when needed, in a sandbox. The size of the buffer depends on complexity and volume. We do not use a fixed percentage**
7. **QA, product and the project lead check that a part is done**

Sources: Tom Cargill, via Jon Bentley, "Programming Pearls", CACM, 1985 · Fred Brooks, "The Mythical Man-Month", 1975

next: Zawinski's law